```
print("Hello, World!")  # This prints a string
```

```
Hello, World!
```

Variables and Data Types

Python is dynamically typed, meaning you don't have to declare the data type

```
# Integer
x = 10
print(x, type(x))

# Float
y = 3.77
print(y, type(y))

# String
name = "FCB"
print(name, type(name))

# Boolean
status = False
print(status, type(status))
```

```
10 <class 'int'>
3.77 <class 'float'>
FCB <class 'str'>
False <class 'bool'>
```

User Input

```
user_name = input("Enter your name : ")
print("Hola,", user_name)
```

```
Enter your name :  VIKKI
Hola, VIKKI
```

Basic Arithmetic Operations

```
a = 10
b = 3

print("Addition:", a + b)
print("Subtraction:", a - b)
print("Multiplication:", a * b)
print("Division:", a / b)
print("Floor Division:", a // b)  # Removes the decimal part
print("Modulus:", a % b)  # Remainder
print("Exponentiation:", a ** b)  # 10^3
```

```
Addition: 13
Subtraction: 7
Multiplication: 30
Division: 3.3333333333333335
Floor Division: 3
Modulus: 1
Exponentiation: 1000
```

Conditional Statements (if-else) in Python

> Conditional statements allow you to execute different blocks of code based on conditions.

Basic if-else Statement

```
age = 18
if age >= 18:
    print("Adult")
else:
    print("Minor")
```

```
Adult
```

```
age = 18

if age <= int(input("Enter your Age:")):
    print("You are an adult.")
else:
    print("You are a minor.")
```

```
Enter your Age: 55
You are an adult.
```

if-elif-else Example

```
marks = 100

if marks >= 90:
    print("Grade: A")
elif marks >= 75:
    print("Grade: B")
elif marks >= 60:
    print("Grade: C")
else:
    print("Grade: F")
```

```
Grade: A
```

Nested if Statement

```
num = 15

if num > 0:
    print("Positive number")
    if num % 2 == 0:
        print("Even number")
    else:
        print("Odd number")
else:
    print("Negative number or zero")
```

```
Positive number
Odd number
```

Short-hand if Statement (Ternary Operator)

```
x = 5
y = 10
min_value = x if x < y else y
print("Smaller number is:", min_value)
```

```
Smaller number is: 5
```

Loops in Python

> Loops are used to execute a block of code multiple times. Python supports for and while loops.

> for Loop Used to iterate over a sequence (list, tuple, string, etc.)

> Example: Iterating over a List

```
# While Loop
i = 0
while i < 5:
    print(i)
    i += 1
```

```
0
1
2
3
4
```

```
# For Loop
for i in range(5):
    print(i)
```

```
0
1
2
3
4
```

```
fruits = ["apple", "banana", "cherry"]

for fruit in fruits:
    print(fruit)
```

```
apple
banana
cherry
```

> Example: Using range() *Function*

> range(start, stop, step) generates numbers in a sequence

```
def greet(name):
    print("Hello", name)

greet("Vikki")
```

```
Hello Vikki
```

```
fruits = ["apple", "banana", "mango"]
print(fruits[2])     # Access
fruits.append("kiwi") # Add
```

```
mango
```

```
for i in range(1, 6):  # Prints numbers from 1 to 5
    print(i)
```

```
1
2
3
4
5
```

```
for i in range(0, 10, 2):  # Prints even numbers (0, 2, 4, 6, 8)
    print(i)
```

```
0
2
4
6
8
```

```
person = {"name": "Vikki", "age": 21}
print(person["name"])  # Access value
```

```
Vikki
```

while Loop

> Executes as long as the condition is True.

> Example: Counting Down

```
count = 5
while count > 0:
    print(count)
    count -= 1  # Decreasing the counter
```

```
5
4
3
2
1
```

while Loop

> Executes as long as the condition is True.

> Example: Counting Down

```
count = 5
while count > 0:
    print(count)
    count -= 1  # Decreasing the counter
```

> Break and continue Statements

>> Break → Exits the loop immediately.

> continue → Skips the current iteration and continues with the next.

> Example: break Statement

```
for num in range(1, 10):
    if num == 5:
        break  # Stops when num reaches 5
    print(num)
```

> Example: continue Statement

```
for num in range(1, 10):
    if num == 5:
        continue  # Skips 5 and continues
    print(num)
```

```
def is_palindrome(text):
    # Remove spaces and convert to lowercase
    cleaned = text.replace(" ", "").lower()

    # Check if the string is equal to its reverse
    return cleaned == cleaned[::-1]

# Example usage
input_text = input("Enter a string: ")

if is_palindrome(input_text):
    print("✅ It's a palindrome!")
else:
```

```
    print("❌ Not a palindrome.")
```

```
Enter a string: HHH
✅ It's a palindrome!
```

OOPS

```
class Car:
    def __init__(self, brand, color):
        self.brand = brand
        self.color = color

    def start(self):
        print(f"{self.color} {self.brand} is starting...")

# Object
my_car = Car("Toyota", "Red")
my_car.start()
```

```
Red Toyota is starting...
```

2. Encapsulation (Hiding Data)

Bind data and methods in one unit (class).

Use _ or __ to make variables protected or private.

```
class Student:
    def __init__(self, name, age):
        self.__name = name  # private variable

    def get_name(self):
        return self.__name

s = Student("Vikki", 21)
print(s.get_name())
```

```
Vikki
```

3. Inheritance (Reusing Code)

One class inherits properties and methods from another.

```
class Animal:
    def speak(self):
        print("Animal speaks")

class Dog(Animal):  # Dog inherits from Animal
    def bark(self):
        print("Dog barks")

d = Dog()
d.speak()
d.bark()
```

```
Animal speaks
Dog barks
```

4. Polymorphism (Same name, different behavior)

a) Method Overriding

```
class Animal:
    def sound(self):
        print("Animal sound")

class Cat(Animal):
    def sound(self):
        print("Meow")

c = Cat()
c.sound()
```

```
Meow
```

b) Duck Typing (Not based on class type, but behavior) python Copy Edit

```
class Bird:
    def fly(self):
        print("Bird flying")

class Airplane:
    def fly(self):
        print("Airplane flying")

def flying_thing(thing):
    thing.fly()

flying_thing(Bird())
flying_thing(Airplane())
```

```
Bird flying
Airplane flying
```

```
class Parent:
    def __init__(self):
        print("Parent constructor")

class Child(Parent):
    def __init__(self):
        super().__init__()
        print("Child constructor")

c = Child()
```

```
Parent constructor
Child constructor
```

Fibonacci

```
# Print Fibonacci sequence up to n terms
n = int(input("How many terms? "))
a, b = 0, 1

print("Fibonacci Sequence:")
for _ in range(n):
    print(a, end=" ")
    a, b = b, a + b
```

```
How many terms? 3
Fibonacci Sequence:
0 1 1
```

```
def fibonacci(n):
    a, b = 0, 1
    for _ in range(n):
        print(a, end=" ")
        a, b = b, a + b

fibonacci(7)
```

```
0 1 1 2 3 5 8
```

```
def fib(n):
    if n <= 1:
        return n
    else:
        return fib(n-1) + fib(n-2)

for i in range(7):
    print(fib(i), end=" ")
```

```
0 1 1 2 3 5 8
```

```
#simple

# Fibonacci Sequence: Simple Code

n = int(input("Enter how many terms: "))
a = 0
b = 1

print("Fibonacci Sequence:")

for i in range(n):
    print(a, end=" ")
    c = a + b
    a = b
    b = c
```

```
Enter how many terms: 9
Fibonacci Sequence:
0 1 1 2 3 5 8 13 21
```

Start coding or generate with AI.